FEAther: A Self-Learning Multi-Agent System for Lightweight Load-Bearing Part Design with Finite Element Analysis

Rifqi Wibisono
Nanjing Xiaozhuang University, Nanjing, China
Student ID L25020014

***Abstract*—We propose to use reinforcement learning to train an open-weight large language model (LLM) to write computer-aided design (CAD) programs for load-bearing parts, rewarded by a finite element analysis (FEA) of the part each program produces. Current text-to-CAD models are trained and judged on whether a shape resembles a reference; whether the part carries its load is never checked. FEAther is a multi-agent system built around one trained agent. An untrained parser turns a request into a structured specification; the trained designer writes a CadQuery program; a sandboxed checker runs it, meshes the part, solves it with CalculiX, and returns a graded reward with a report. The system is self-learning: the designer is trained on its own scored attempts, and at run time it revises its design from the report until the part passes, then removes material while it still passes. We ask whether an FEA-graded reward beats a shape-similarity reward (RQ1), whether run-time feedback adds anything beyond training or only adds attempts (RQ2), and how much material the passing designs use (RQ3). A pilot has built the oracle and the data. The checker's FEA agrees with hand calculations within 10% on ten reference parts. Forty generated specifications are each proven solvable. The untrained Qwen3-1.7B and Qwen3-8B models solve none of them, and repeat their own failed program in about 80% of revision attempts.**

***Index Terms*—code generation, CAD, finite element analysis, reinforcement learning, self-repair, test oracle**

I. Introduction

A parametric CAD model can be written as a program. In CadQuery [1], a Python library on the OpenCASCADE geometry kernel, a bracket is a script that sketches profiles, extrudes them and cuts holes; running it yields a solid that can be meshed, simulated and printed. Because such programs are text, LLMs can learn to write them, and recent models translate text, images or point clouds into CadQuery [2]–[4].

These models are trained and evaluated on geometric resemblance: a reward or metric compares the generated shape with a reference shape, for example by Chamfer distance. For a load-bearing part, resemblance is the wrong oracle. Two brackets that look alike can differ widely in peak stress, because stress concentrates at small features such as the inside radius of a bend or the edge of a hole [5], [6]. Regenwetter et al. showed that similarity metrics miss engineering performance [7]. When designs are checked by physics, current systems fail: in Son et al.'s 50 FEA-checked engineering briefs, no frontier agent's first design passed every requirement [8], and CADEngBench reports that matching a reference FEA response "remains difficult" [9].

From a software-engineering view, a CAD program is code whose test oracle is physics. Code-generation research offers two lessons that should transfer: execution feedback can train a model through reinforcement learning, and feedback at run time (self-repair) helps less than expected once its extra attempts are paid for [10]. Neither has been tested with a physics oracle.

The closest work, COSMO-Agent [11], trains Qwen3-8B with the GRPO algorithm and the CalculiX solver in the loop, but only to choose parameter values for 25 fixed CadQuery templates. In engineering terms this is *sizing*; it caps cost rather than minimising mass, and it used 16 H200 GPUs. No work we found trains a model to make *configuration* choices (the layout: where material goes, whether there is a rib or a cutout), written as a CAD program, against FEA-checked requirements. Our last arXiv search, on 1 November 2026, found RL-trained CAD generators that still reward only shape resemblance. This proposal contributes:

- an FEA test oracle for generated CAD programs, with sandboxed execution and a staged reward, validated against hand calculations;
- a specification generator in which every specification is proven solvable, and a sizing-only baseline that separates configuration from sizing;
- a recipe that trains one designer to design, revise and lighten on a single 16 GB GPU;
- an evaluation that separates the effect of feedback from the effect of more attempts.

II. Related Work

LLMs that write CAD programs. CAD-Recode reconstructs CadQuery programs from point clouds, trained on one million procedurally generated programs [2]. CAD-Coder generates CadQuery from text, fine-tuned on about 110,000 text–code–shape triplets and then trained with GRPO on a geometric reward [3]; cadrille applies reinforcement learning to multi-modal reconstruction [4]. All of their rewards measure resemblance to a target shape.

Physics in the loop. Son et al. give frontier agents FEA reports at run time and let them iterate; their agents are called self-improving because they iterate on feedback, but no model is trained [8]. CADEngBench benchmarks parametric design against simulation without training [9]. COSMO-Agent trains

TABLE I
RESEARCH QUESTIONS, VARIABLES AND ANALYSIS

Question	Independent variable	Dependent variable	Analysis
RQ1: Does GRPO with an FEA-graded reward raise the share of held-out specs fully passed, compared with SFT alone and with a shape-similarity reward?	training: SFT only; GRPO with a shape reward, an FEA reward, or both	full pass rate; share of requirements met; invalid rate	McNemar's test; bootstrap 95% intervals
RQ2: Does run-time FEA feedback improve designs beyond what training put into the model, and is the gain from the feedback or from more attempts?	designer: SFT, GRPO, GRPO with revise and lighten prompts; attempts: one-shot, best-of-k, loop; $k \in \{1, 3, 5, 10\}$	specs passed within k; mass of the lightest pass; tokens per solved spec	McNemar's test at each k
RQ3: How much material do passing designs use, and what does the reward's mass term change?	reward with or without the mass term	mass relative to sizing-only and topology-optimised references; pass rate	Wilcoxon signed-rank test

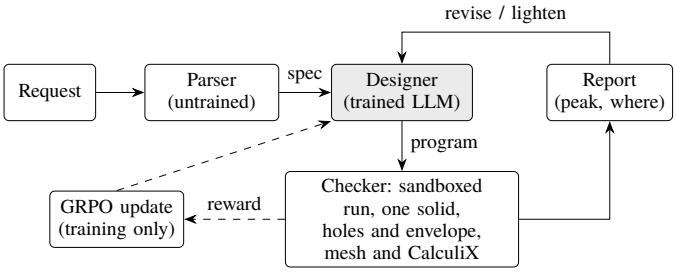


Fig. 1. FEAther. Only the designer is trained; the parser and the report stay outside the reward path. At run time the report drives revisions; in training the checker's reward updates the designer.

on simulation results but sizes fixed templates [11]. FEAther differs from all three by training the model that chooses the configuration, and by rewarding low mass as well as passing.

Self-repair in code generation. Self-Debug [12] teaches models to fix their code from execution results. Olausson et al. found that, at an equal budget, self-repair often gains little over drawing more independent samples, because models rarely act well on feedback [10]. RQ2 tests this claim with a physics oracle, before and after the repair skill is trained.

III. Research Questions and Hypotheses

Table I lists the questions, their variables and the planned analysis. The hypotheses are: **H1:** the FEA-graded reward raises the full pass rate over both SFT alone and the shape reward, because only it scores what a spec requires. **H2:** for untrained and SFT-only designers, best-of-k sampling matches or beats the loop at equal k, as in our pilot; after training on revise and lighten prompts, the loop beats best-of-k. **H3:** without the mass term, passing designs are no lighter than the sizing-only reference; with it, they are lighter, because the designer can move material instead of only scaling it.

IV. Approach

Specifications. Each specification (spec) is a structured record and an English prompt that states every number in it: a load (force, point, direction and patch radius), bolt holes (axis and diameter), an envelope the part must stay inside, the material (PLA, Young's modulus 2300 MPa) and limits on peak stress, deflection and mass. A generator samples specs for each part family (first L-brackets and wall hooks, about five by the end, with one or two families held out of training). It sizes a reference program from that family's template by hand calculation and keeps the spec only if the checker passes the reference. Each kept spec is therefore proven solvable. The reference then sets two limits. The deflection limit is 1.3 to 2 times the reference's deflection. The mass budget is 80% of the reference's mass, because a budget at the reference would stop rewarding any design lighter than what sizing alone achieves.

Checker. Programs from the model are untrusted. Each one runs in its own process with no network (its own user and network namespaces) and with limits on memory, CPU time and file size; it must leave its part in a variable named `result`. The checker then grades in stages, and credit stops at the first that fails: the program runs (0.1); it makes exactly one valid solid (0.1); every bolt hole is open where the spec puts it, a surface lies at the load point and the part stays in the envelope (0.2); the structure (0.4, the mean over requirements of $\min(1, \text{limit}/\text{value})$ for peak stress and deflection); and the mass ($0.2\,\min(1, \text{budget}/\text{mass})$, counted only once the structure passes). Gmsh [13] meshes the part with quadratic tetrahedra, refined on curved faces, and CalculiX [14] solves a linear static case: bolt bores held rigid, the load spread over a patch. Both of these raise stress artificially near the bores and the patch, so peak von Mises stress is read outside a zone one bolt diameter beyond each bore and twice the patch radius round the load. The report states which stage failed, the peak stress, where it is and by how much it misses its limit (Fig. 2).

Agents and loop. Fig. 1 shows the system. The designer writes a program; while it fails, the designer receives its own program and the report and revises; once it passes, it is asked for a lighter version that still passes, and the lightest passing design is kept.

Training. The designer is Qwen3-8B [15], with Qwen3-1.7B for pilots. Supervised fine-tuning (SFT) on public text-to-CadQuery data [2], [3] teaches the language. Group Relative Policy Optimization (GRPO) [16] then samples a group of $G = 8$ programs for each prompt, scores them with the checker

(a) Prompt of a generated spec. I need a wall bracket that holds a load out from the wall, printed in PLA. Stay within the box x 0 to 60, y -15 to 15, z -1 to 76 mm. Requirements: peak stress under 10 MPa, deflection under 3.8 mm, and mass ideally below 9.3 g. Mount it on the wall at x = 0 with 2 M5 bolts; their 5.5 mm holes run along x at (y, z) = (0, 10) and (y, z) = (0, 43) mm. A 10 N load presses straight down on it at the point (42, 0, 76) mm, spread over a patch of radius 4 mm.

(b) Checker report on reference part L1, built to fail.

```
FAIL at stage 'structure' (reward 0.70 of 1.00)
- Largest deflection 3.90 mm (no limit set).
- Peak stress 13.36 MPa at (6.6, 0.9, 61.4) mm;
  limit 10.00 MPa, over the limit by 34%.
- Mass 24.38 g (not scored until the structure passes).
```

Fig. 2. What the designer sees: (a) a spec's prompt, with every number in the spec; (b) the report it revises from. The peak lies on the inside of L1's 2 mm bend radius, where curved-beam theory puts it.

TABLE II
CHECKER FEA AGAINST HAND CALCULATIONS (PEAK STRESS)

Family	Parts	Hand calculation at worst point	FEA / hand
L-bracket	4	curved beam (Winkler), bend	1.046–1.096
Wall hook	3	curved beam (Winkler), curve	0.975–1.047
Hanger strap	3	hole in a strip (Heywood)	1.013–1.024

$(r_1, \ldots, r_G)$ and gives each program the advantage

$$A_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}. \quad (1)$$

The update makes the tokens of programs with $A_i > 0$ more likely and the rest less likely. A prompt whose eight programs all score the same therefore teaches nothing, which is why the reward is graded rather than pass/fail. Training mixes three prompt kinds, the same ones the loop uses: design; revise, with one of the model's own failed programs and its report; and lighten, with a passing program. QLoRA [17] freezes the base model in 4-bit precision and trains small adapter matrices, which brings the 8B model within a 16 GB laptop GPU (about 11 GB, estimated). FEA rewards run on 20 CPU workers at 2–7 s each, so generating designs on the GPU is the bottleneck.

V. Preliminary Results

All results below come from the project repository and can be re-run with one command each.

The oracle agrees with theory. A cantilever beam gave a tip deflection of 1.736 mm against 1.739 mm from beam theory. Ten hand-written reference parts in three families put their worst point where a reliable formula applies (Table II): curved-beam theory for the bends of L-brackets and hooks, and Heywood's formula for a hole in a strip [5], [6]. On all ten, the checker's FEA peak lies within 10% of the hand calculation and within 8 mm of the predicted point or its mirror image, and refining the mesh moves each value by under 4%. The checker fails the three parts built to exceed the 10 MPa limit and passes the other seven, in 1.6–7.3 s per part. Separate tests confirm that a crashing program, an endless loop, a program grabbing 8 GB, an attempt to reach the network, a part in two pieces, a missing bolt hole and a hole 0.7 mm too large each fail at the right stage.

Every spec is solvable. The generator produced 40 specs, 20 L-brackets and 20 hooks, in 104 s on eight workers; every reference passed on its first FEA run, with peaks of 7.6–9.1 MPa against the 10 MPa limit.

TABLE III
UNTRAINED BASELINE: SPECS (OF 40) WHOSE BEST OF EIGHT ATTEMPTS GOT PAST EACH STAGE

	Qwen3-1.7B		Qwen3-8B	
Stage passed	samples	loop	samples	loop
program runs	8	1	34	18
one valid solid	5	1	12	4
holes, envelope	0	0	0	0
all requirements	0	0	0	0

Untrained models solve nothing. We served Qwen3-1.7B (16-bit) and Qwen3-8B (8-bit floating point) with vLLM [18], thinking mode off, and gave each eight checker runs per spec, either as eight independent samples or in the loop (Table III). Neither model passed a spec. Most programs misuse the CadQuery interface; those that make a solid cross into the wall, because CadQuery centres boxes on the origin. In the loop, about 80% of revisions (225 of 280 for 1.7B, 221 of 280 for 8B) repeat the program they were asked to fix, so independent samples reach running code on more specs than the loop does (34 against 18 for the 8B). This agrees with Olausson et al. [10] and is the baseline that trained revision must beat. GRPO would already have a signal on 34 of 40 specs, where the 8B's samples score differently, but only between the credits for running, for one solid and for the interfaces; no design reached the structural checks. SFT therefore comes first.

VI. Evaluation Plan

Test material. Held-out specs, including whole held-out families, and Son et al.'s briefs if they are released [8].

Baselines. The untrained model; the SFT-only model; SFT plus GRPO with a shape-similarity reward, as in CAD-Coder [3]; and a sizing-only search that tunes the reference template's parameters without an LLM. The last isolates what choosing the configuration adds over sizing, the difference from COSMO-Agent.

Measures. The full pass rate (share of specs meeting every requirement); the share of requirements met; the invalid rate (programs that crash or make no single solid); mass relative to the references; the pass rate within k attempts; and tokens and seconds per solved spec.

Statistics. Every condition attempts the same specs, so pass/fail outcomes are paired and compared with McNemar's test, which counts only the specs on which two conditions disagree. Rates carry bootstrap 95% intervals, and mass ratios use the Wilcoxon signed-rank test, which does not assume normally distributed differences. As an extension, a subset of before-and-after designs will be 3D-printed and broken to compare measured failure loads with FEA.

VII. Threats to Validity

Construct. The oracle is a simplified model: linear static, rigid bolts, no contact, and an isotropic material, while printed

PLA is weaker across its layers. Validation against hand calculations and the planned physical tests bound this. *Internal.* A trained model may exploit the oracle, for example with geometry that meshes oddly; the staged checks, the stress-reading zones, mesh-refinement spot checks and a log of every exploit found address this. *External.* The specs come from our own generator; held-out families test whether the model generalises beyond it, and the first results cover only PLA and up to five families. *Conclusion.* The 40-spec pilot is small; the final evaluation uses hundreds of held-out specs and repeated training seeds.

VIII. Risks, Plan and Availability

Risks and fallbacks. If SFT still leaves no designs reaching the structural checks, training starts from easier specs (limits well above the reference's stress) and from revise prompts built by breaking reference programs. If GRPO on the 8B model does not fit in 16 GB, the 1.7B model is trained fully and a rented GPU runs the 8B experiments. If the checker is exploited, it is fixed and earlier checkpoints are re-scored with the fixed version. If a newer paper trains a CAD generator on FEA results, the headline moves to the print-aware reward and the physical tests.

Expected outcomes. Success means a significant gain in full pass rate over SFT alone and over the shape reward (H1), a loop that beats best-of-k once revision is trained (H2), and passing designs lighter than sizing alone achieves (H3). A negative result on H2 would still be informative: it would extend Olausson et al.'s finding [10] from code to physics-checked design.

Plan. Months 1–2: SFT, then a GRPO pilot on the 1.7B model and the 8B model. Months 3–4: five families and the experiments for RQ1–RQ3. Month 5: physical tests. Month 6: a demonstration and the thesis.

Availability. Code, the 40 specs and all baseline results are in a Git repository that will be made public with the thesis, and is available to reviewers on request.

References

[1] CadQuery contributors, "CadQuery: A Python parametric CAD scripting framework based on OCCT," https://github.com/CadQuery/cadquery, 2026, version 2.8.

[2] D. Rukhovich, E. Dupont, D. Mallis, K. Cherenkova, A. Kacem, and D. Aouada, "CAD-Recode: Reverse engineering CAD code from point clouds," arXiv:2412.14042, 2024.

[3] Y. Guan, X. Wang, X. Xing, J. Zhang, D. Xu, and Q. Yu, "CAD-Coder: Text-to-CAD generation with chain-of-thought and geometric reward," in *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2025, pp. 59 765–59 789.

[4] M. Kolodiazhnyi, D. Tarasov, D. Zhemchuzhnikov, A. Nikulin, I. Zisman, A. Vorontsova, A. Konushin, V. Kurenkov *et al.*, "cadrille: Multi-modal CAD reconstruction with reinforcement learning," in *International Conference on Learning Representations (ICLR)*, 2026.

[5] R. G. Budynas and J. K. Nisbett, *Shigley's Mechanical Engineering Design*, 11th ed. McGraw-Hill, 2020.

[6] W. D. Pilkey and D. F. Pilkey, *Peterson's Stress Concentration Factors*, 3rd ed. Wiley, 2008.

[7] L. Regenwetter, A. Srivastava, D. Gutfreund, and F. Ahmed, "Beyond statistical similarity: Rethinking metrics for deep generative models in engineering design," *Computer-Aided Design*, vol. 165, p. 103609, 2023.

[8] G. Son, J. Park, S. Park, S. Ahn, and Y. Yu, "Self-improving CAD generation agents with finite element analysis as feedback," arXiv:2605.17448, 2026.

[9] H. Singh, A. Dubey, and J. A. A. Herrera, "CADEngBench: It looks like CAD, but does it work? evaluating parametric design, assembly reasoning, and physics simulation," arXiv:2608.09296, 2026.

[10] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, and A. Solar-Lezama, "Is self-repair a silver bullet for code generation?" in *International Conference on Learning Representations (ICLR)*, 2024.

[11] L. Deng, S. Deng, Y. Chen, Y. Dai, Z. Zhong, L. Li, X. Sun, Y. Shi *et al.*, "COSMO-Agent: Tool-augmented agent for closed-loop optimization, simulation, and modeling orchestration," arXiv:2604.05547, 2026.

[12] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *International Conference on Learning Representations (ICLR)*, 2024.

[13] C. Geuzaine and J.-F. Remacle, "Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities," *International Journal for Numerical Methods in Engineering*, vol. 79, no. 11, pp. 1309–1331, 2009.

[14] G. Dhondt, *The Finite Element Method for Three-Dimensional Thermomechanical Applications*. Wiley, 2004.

[15] A. Yang *et al.*, "Qwen3 technical report," arXiv:2505.09388, 2025.

[16] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang *et al.*, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," arXiv:2402.03300, 2024.

[17] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.

[18] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.